

Unbounded Knapsack

Complete Documentation

Problem Statement

You are given:

- $wt[]$ = weights of items
- $val[]$ = values of items
- W = maximum capacity of knapsack

You can take an item:

- 0 times
- 1 time
- 2 times
- 3 times
- ...
- unlimited times

Goal: Maximize the total value without exceeding capacity W .

Example

$wt = [2,4,6]$

$val = [5,11,13]$

$W = 10$

Possible selections:

- $2 + 2 + 2 + 2 + 2$
value = 25
- $4 + 4 + 2$
value = 27
- $6 + 2 + 2$
value = 23

Best answer: 27

Key Observation

This is an:

Optimization Problem

Unlike Coin Change II:

- Count all ways

here we need:

- Maximum Value

Therefore:

```
return max(notTake, take);
```

instead of:

```
return take + notTake;
```

Recursive State Definition

Define:

$f(ind, W)$

Meaning:

- Maximum value obtainable using items from index 0...ind with remaining capacity W.

Example:

$f(2,10)$ means Maximum value obtainable using items {0,1,2} with capacity 10.

Choices At Every Item

At every item:

- Take
- Not Take

Choice 1 : Not Take

Skip current item.

```
notTake = f(ind-1, W)
```

Move to previous item.

Choice 2 : Take

If item fits:

```
wt[ind] <= W
```

take it:

```
take = val[ind] + f(ind, W-wt[ind])
```

Notice:

- **index remains same** because item can be reused infinitely.

This is the most important property of: **Unbounded Knapsack**

Recurrence Relation

```
f(ind,W) = max( f(ind-1,W), val[ind] + f(ind,W-wt[ind]) )
```

Base Case

Your code uses:

```
if(ind == 0)
{
    return (W / wt[0]) * val[0];
}
```

Understanding The Base Case

Suppose:

- $wt[0] = 2$
- $val[0] = 5$
- $W = 10$

Only one item remains.

Question: How many copies of weight 2 can fit inside capacity 10?

Answer: $10 / 2 = 5$ copies

Value: $5 \times 5 = 25$

Return: $(10/2)*5 = 25$

Example 2

- $wt[0] = 3$
- $val[0] = 7$
- $W = 10$

Copies possible: $10/3 = 3$

Value: $3 \times 7 = 21$

Return: 21

Why This Base Case Works

At $ind = 0$ only one item remains. Since repetition is allowed, take as many copies as possible.

Therefore: $(W / wt[0]) * val[0]$ gives the optimal answer immediately.

Complete Recursive Code

```
int solve(int ind,
          int W,
          vector<int>& wt,
          vector<int>& val)
{
    if(ind == 0)
    {
        return (W/wt[0])*val[0];
    }

    int notTake =
        solve(ind-1,W,wt,val);

    int take = INT_MIN;

    if(wt[ind] <= W)
    {
        take =
            val[ind]
            +
            solve(ind,
                  W-wt[ind],
                  wt,val);
    }

    return max(notTake,take);
}
```

Recursion Tree

Input:

wt = [2,4,6]

val = [5,11,13]

W = 10

Start: $f(2,10)$

Tree

```
f(2,10)
├── notTake
│   └── f(1,10)
│       ├── notTake
│       │   └── f(0,10)
│       │       = 25
│       └── take
│           11 + f(1,6)
└── take
    13 + f(2,4)
```

Expand: $f(1,6)$

```
f(1,6)
├── notTake
│   └── f(0,6)
│       = 15
└── take
    11 + f(1,2)
```

Observe: Take branch stays on same index. This is the signature pattern of: **Unbounded Knapsack**

Memoization

Overlapping states:

- $f(1,6)$
- $f(1,2)$
- $f(0,10)$

- f(0,6)

are recomputed many times.

Store them:

```
vector<vector<int>> dp(n, vector<int>(W+1, -1));
```

Memoization Code

```
int solve(int ind,
          int W,
          vector<int>& wt,
          vector<int>& val,
          vector<vector<int>>& dp)
{
    if(ind == 0)
        return (W/wt[0])*val[0];

    if(dp[ind][W] != -1)
        return dp[ind][W];

    int notTake =
        solve(ind-1, W, wt, val, dp);

    int take = INT_MIN;

    if(wt[ind] <= W)
    {
        take =
            val[ind]
            +
            solve(ind,
                  W-wt[ind],
                  wt, val,
                  dp);
    }

    return dp[ind][W]
        =
        max(notTake, take);
}
```

Converting Memoization To Tabulation

Memoization State: dp[ind][W] becomes dp[i][cap]

Step 1 : Create Table

```
vector<vector<int>> dp(n, vector<int>(W+1,0));
```

Step 2 : Fill Base Row

Memoization base: $(W/wt[0]) * val[0]$

Convert into:

```
for(int cap=0; cap<=W; cap++)  
{  
    dp[0][cap] = (cap/wt[0]) * val[0];  
}
```

Why? At row 0: Only item 0 exists. Use as many copies as possible.

Step 3 : Fill Remaining Rows

Memoization:

```
notTake = dp[i-1][cap]
```

```
take = val[i] + dp[i][cap-wt[i]]
```

Notice: **same row** because items are reusable.

Tabulation Code

```
for(int i=1;i<n;i++)
{
    for(int cap=0;
        cap<=W;
        cap++)
    {
        int notTake
            =
            dp[i-1][cap];

        int take
            =
            INT_MIN;

        if(wt[i] <= cap)
        {
            take =
                val[i]
                +
                dp[i]
                [cap-wt[i]];
        }

        dp[i][cap]
            =
            max(notTake, take);
    }
}
```


Final Tabulation Solution

```
int unboundedKnapsack(
    int n,
    int W,
    vector<int>& val,
    vector<int>& wt)
{
    vector<vector<int>>
    dp(n,
        vector<int>(W+1,0));

    for(int cap=0;
        cap<=W;
        cap++)
    {
        dp[0][cap]
        =
        (cap/wt[0])*val[0];
    }

    for(int i=1;i<n;i++)
    {
        for(int cap=0;
            cap<=W;
            cap++)
        {
            int notTake
                =
                dp[i-1][cap];

            int take
                =
                INT_MIN;

            if(wt[i] <= cap)
            {
                take =
                    val[i]
                    +
                    dp[i]
                    [cap-wt[i]];
            }

            dp[i][cap]
            =
            max(notTake, take);
        }
    }

    return dp[n-1][W];
}
```

Space Optimization

Observation: Current row depends on Previous row and Current row.

Therefore: 2 arrays sufficient.

Conversion

```
notTake = prev[cap]
```

```
take = val[i] + curr[cap-wt[i]]
```

Notice: **curr** because we stay on same row. This is exactly like **Coin Change II** where: `take = curr[target-coin]`

Space Optimized Code

```
vector<int> prev(W+1,0);

for(int cap=0; cap<=W; cap++)
{
    prev[cap] = (cap/wt[0]) * val[0];
}

for(int i=1;i<n;i++)
{
    vector<int> curr(W+1,0);

    for(int cap=0; cap<=W; cap++)
    {
        int notTake = prev[cap];
        int take = INT_MIN;

        if(wt[i] <= cap)
        {
            take = val[i] + curr[cap-wt[i]];
        }

        curr[cap] = max(notTake,take);
    }
    prev = curr;
}

return prev[W];
```

Pattern Recognition

0/1 Knapsack	Unbounded Knapsack
Take: <pre>take = value + dp[i-1][W-weight]</pre> Move: Previous Row because item can only be used once.	Take: <pre>take = value + dp[i][W-weight]</pre> Move: Current Row because item can be used infinitely.

Quick Rule

Whenever you see:

- Unlimited Supply
- Infinite Reuse
- Can Pick Again

Think: **Take Branch Stays On Same Index**

Formula:

```
take = value + f(ind, W-weight)
```

instead of:

```
take = value + f(ind-1, W-weight)
```

That single difference is what converts **0/1 Knapsack** → **Unbounded Knapsack** and is the most important concept in the entire problem.